Presented with xmind

处理机管理

进程

- 概念分
  - 进程 程序 进程映像
    - 程序
      - 静态的
      - 联系
      - 进程是程序的一次执行过程
      - 同一个程序执行很多次会有很多个进程
      - 进程映像
    - 动态的
      - 联系
      - 一个进程在某一时刻的快照就叫做进程映像
    - 进程
      - 联系
      - 进程映像
- 组成
  - PCB（Process Control Block）组成
    - PID process identifier
      - 进程被创建的时候操作系统会为进程分配一个唯一的不重复的身份证号PID
      - 进程描述信息
    - UID user identifier
      - 进程所属用户的ID
      - 进程描述信息
    - 记录分配了哪些资源
      - 分配了多少内存，正在使用哪些I/O设备，正在使用哪些文件，可以用于操作系统对资源的管理
      - 资源分配清单
    - 记录进程的运行情况
      - CPU使用时间
      - 磁盘使用情况
      - 网络流量使用情况
      - ……
      - 进程当前的状态
        - 就绪态
        - 阻塞态
        - 运行态
        - 创建态
        - 终止态
        - 挂起态
          - 阻塞挂起态
          - 就绪挂起态
      - 进程控制和管理信息
      - 这个是给操作系统管理用的
    - 处理机相关信息
      - PSW PC等等相关寄存的值
      - 用于实现进程之间的切换
      - 处理机相关信息
    - PCB的创建和回收
      - 进程创建的时候，PCB被创建
      - 进程终止的时候，PCB被回收
  - 调用栈
  - 程序段
    - 程序的代码
  - 数据段
    - 运行过程中产生的各种数据
    - 这个是给进程自己用的与自身逻辑有关
- 特征
  - 动态性
    - 动态性是进程最基本的特征
  - 并发性
  - 独立性
  - 异步性
    - 异步性会导致开发程序执行结果不确定性
  - 结构性
- 进程的状态和转换
  - 进程的状态
    - 创建态
    - 就绪态
    - 运行态
    - 阻塞态
    - 终止态
    - 转换关系
      - 具体的转换关系
        - 运行态→终止态：当一个进程到达了自然结束点，或是出现了无法克服的错误（终止），或是被操作系统所终结，或是被其他终止权的进程所终结。
        - 运行态→就绪态：运行时间片到，出现有更高优先级的进程。
        - 运行态→阻塞态：等待使用资源；如等待外设传输；等待人工干预。
        - 就绪态→终止态：未在状态转换图中显示，但某些操作系统允许父进程终结子进程。
      - 没有进程调度的时候，不可能只有就绪态的进程，而没有运行态的进程
- 进程的组织
  - 链接方式
    - 按照进程状态将PCB分为多个队列
  - 索引方式
    - OS持有指向各个队列的指针
    - 根据进程状态的不同，建立几张索引表
    - OS持有指向各个索引表的指针
- 进程控制
  - 功能
    - 对系统中所有进程实施有效的管理
      - 创建新进程
      - 撤销已有进程
      - 实现进程状态转换
  - 如何实现进程控制
    - 通过原语来实现
      - 原语的执行具有原子性，期间不允许被中断
      - 原语的实现是通过开关中断来实现的
      - 进程的控制这块就知道他是原语实现的就行了
  - 有什么原语
    - 创建原语
    - 撤销原语
    - 进程的阻塞和唤醒的原语
    - 进程切换的原语
  - 原语做了什么
    - 更新PCB中的信息
    - 将PCB插入合适的队列
    - 分配/回收资源
- 进程通信
  - 为什么要有进程通信
    - 进程是分配系统资源的基本单位，因此各个进程拥有的内存地址空间相互独立
    - 为了保证安全一个进程不能直接访问另一个进程的地址空间
  - 共享存储
    - 基于存储区的共享
      - 数据的形式、存放位置都是由通信进程控制的
      - 高级通信方式
    - 基于数据结构的共享
      - 比如，共享空间里只能放一个长度为10的数组
      - 低级通信方式
  - 进程通信
    - 消息传递
      - 直接通信方式
        - 消息发送进程要指明接收进程的ID
      - 间接通信方式
        - 通过信箱间接地通信，因此又称为“信箱通信方式”
    - 管道通信
      - 管道是一个特殊的共享文件，又叫做PIPE文件，其实就是在内存中开辟一个大小固定的缓冲区
        - 半双工
        - 互斥访问
        - 管道写满
          - 写进程阻塞
        - 管道读空
          - 读进程阻塞
      - 注意管道中的数据一旦被读出来，数据就知道消失了，多个进程读同一个管道的时候可能会错乱
        - 解决办法1：多个写进程一个读进程
        - 解决办法2：多个写进程，多个读进程，轮流读数据
      - 管道程序是通过内核缓冲区实现的进程间通信机制，分为匿名管道（pipe()）和命名管道（mkfifo）
- fork函数
  - 为当前所有的进程创建一个子进程

线程

- 什么是线程
  - 有的进程可能需要同时做很多事，传统的进程只能串行执行一系列程序，因此才需要引入线程来增加并发度
- 线程属性
  - 线程变成处理机调度的基本单位了
    - 线程几乎不拥有系统资源
    - 但是资源分配的基本单位还是进程
    - 同一进程的不同线程之间共享进程的资源
    - 不同进程之间的线程切换，会引起进程切换，进程切换开销就很大了
    - 同进程内的线程切换，系统开销小
  - 多个CPU计算机中，各个线程可以占用不同的CPU，这也就说明一个进程可以占用多个CPU
  - 每个线程都有一个线程ID 线程控制块TCB
  - 线程也会有就绪阻塞运行三种状态
  - 由于共享内存的存储空间，同一进程中的线程之间的通信甚至无需系统干预
- 线程实现方式
  - 用户级线程
    - 线程库实现
      - 比如while循环中套了三个语句，那么这个while就是一个线程库，一个叫做一个线程
      - 但是就是不管你用户级线程实现的再花花，你从操作系统来看，你都是一个进程给你分配的只是一个CPU
  - 用户级线程是代码逻辑的概体
  - 内核级线程
    - 内核支持的线程
      - 内核级线程的管理工作由操作系统内核完成的
      - 线程调度、切换工作都是由内核负责的，因此内核及线程的切换必然需要在核心态才能完成
      - 操作系统会为每个内核级线程建立相应的TCB，通过TCB对线程进行管理，内核级线程就是从操作系统内核视角看到的线程
      - 缺点就是操作系统实现线程级内核切换，开销很大
      - 线程类型
        - 一对一
        - 多对一
        - 多对多 n对m n>m
        - 每个线程对应相应的内核级线程，也就是说用的是不同的CPU，如果是单想要实现线程的切换，必须要交给操作系统内核来实现，这也是开销大的来源
  - 内核级线程是运行机会的概体
- TCB
  - 线程标识符
  - 程序计数器PC
  - 其他寄存器
  - 堆栈指针
  - 线程运行状态
  - 优先级
- 线程表
  - 可以将多个TCB组织成一张线程表

系统调用

- 中断和异常
  - 中断
    - 为什么要有中断这个东西，中断的一个用途
      - 如果是的CPU的使用权从操作系统进给用户的话，那么这个信赖作系统自己主动移交CPU就可以实现
      - 但是如果是操作系统想要从用户手里拿回CPU的使用权，就只能通过中断的方法
    - 中断的类型
      - 内中断，又叫做异常
        - 当前CPU执行的指令发出的中断
        - 故障
          - 错误条件引起的，可以被修复的
          - 0.缺页故障
        - 终止
          - 致命错误引起内核程序无法修复该错误，直接终止该应用程序
          - 1.用户态执行特权指令
          - 2.除数是0
        - 3.用户程序希望请求操作系统的服务
      - 外中断
        - 时钟中断
          - 时间片用完
        - I/O中断
          - 输入输出任务完成的时候，向CPU发送中断信号
    - 中断机制的基本原理
      - 不同的中断信号，需要不同的中断处理程序来处理，当CPU检测到中断信号之后，会根据其类型，查询中断向量表，以此来找到应该中断处理程序在内存中的位置
- 系统调用
  - 系统调用的定义
    - 系统调用是操作系统提供给应用程序的特殊函数，应用程序可以通过系统调用来获得操作系统内核的服务，系统调用是操作系统给程序员提供的接口
  - 系统调用的分类（按照功能）
    - 设备管理
      - 完成设备的请求释放启动等等
    - 文件管理
      - 完成文件的读写创建删除
    - 进程控制
      - 完成进程的创建撤销阻塞唤醒等功能
    - 进程通信
      - 完成进程之间的消息传递 信号传递等功能
    - 内存管理
      - 完成内存的分配回收等功能
  - 系统调用的过程
    - 传递系统调用参数
    - 执行陷入指令（用户态）
      - 陷入指令又称为访管指令，trap指令
    - 执行相应的内存请求和程序处理系统调用（核心态）
    - 返回应用程序

虚拟机

- 第一类VMM
  - 直接运行在硬件上
  - 分配的是未经抽象的物理硬件
- 第二类VMM
  - 运行在宿主操作系统上
  - 分配的是经过抽象的资源
  - 比如虚拟内存等等
- 裸机+操作系统就是一个虚拟机
- 虚拟机：使用虚拟化技术，将一台物理机器虚拟化为多台虚拟机器，每个虚拟机器都可以独立运行一个操作系统

死锁

- 概念
  - 死锁是指多个进程（或线程）在执行过程中，因争夺资源而陷入互相等待的状态，导致所有进程都无法继续执行的情况
- 产生
  - 必要条件
    - 互斥
      - 只有对必须互斥使用的资源的争抢才会导致死锁，诸如：内存、磁盘驱动器、打印机等
    - 不剥夺
      - 进程所获得的资源在未使用之前，不能由其他进程强行夺走，只能主动释放
    - 请求和保持
      - 进程已经保持了至少一个资源，但又提出了新的资源请求，而该资源又被其他的进程所占有，此时请求进程被阻塞，但又对自己已有的资源保持不放
    - 循环等待
      - 存在一种进程资源的循环等待链，链中的每一个进程已获得的资源同时被下一个进程所请求
- 预防
  - 破坏互斥
    - 如果把只能互斥使用的资源改造成允许共享使用的，那么系统就不会进入死锁状态
    - 缺点：并不是所有的互斥资源都可以改成共享使用的
    - 增加一个中间件
      - 比如打印机，一个时间段内只能打一个进程的数据，但是如果中间加了一台缓冲区，多个进程可以同时向里输入自己要打印的内容，实现共享使用
  - 破坏不剥夺
    - 方案1
      - 当某个进程请求新的资源得不到满足的时候，它必须立即释放保持的所有资源，待以后需要的时候再重新申请。也就是说，即使某些资源尚未使用完也需要主动释放
      - 缺点
        - 实现复杂
        - 释放已经获得的资源一般只适用于易于恢复状态的资源
        - 反复地申请和释放资源会增加系统开销，降低系统吞吐量
        - 如果采用方案1，如果暂时地得不到某个资源之前获得的全部资源都要释放掉，如果一直发生这种的情况会导致饥饿
    - 方案2
      - 当某个进程需要的资源得不到满足的时候，可以由操作系统协助，将想要的资源强行剥夺，这种方式一般需要考虑进程的优先级
  - 破坏请求与保持条件
    - 可以采用静态分配的方法
    - 一次性分配全部需要的资源
      - 缺点
        - 缺点资源利用率较低，一旦占有就一直占有，CPU运行的时候，也基于不了其他事，效率很低
        - 实现比较简单
  - 破坏循环等待条件
    - 顺序资源分配的方法
      - 1.给系统的资源编号
      - 2.每个进程必须按照编号递增的顺序请求资源，同类资源一次申请完
      - 原理就是不给进程申请到的资源的编号比我小，它可以申请我的资源，但是我的编号已经过了他的资源的编号，那么我就不能申请他的资源了，从而打破了循环等待的过程
      - 缺点
        - 不方便增加新的设备，因为可能会把所有设备的编号全部修改
        - 进程实际使用资源的顺序可能和编号递增顺序不一致，导致资源的浪费
        - 必须按照规定次序申请资源，用户编程麻烦
- 避免
  - 安全序列
    - 就是指如果系统按照这种序列分配资源，则每个进程都能顺利完成。只要能找出一个安全序列，系统就是安全状态。当把安全序列写错很多个
    - 一定不会发生死锁
  - 不安全状态
    - 可能会发生死锁
    - 分配了资源之后，如果找不到任何一个安全序列，那么当前就处于不安全状态。这就意味着所有进程都无法顺利的执行下去
    - 但是如果有一些进程能顺利结束了资源，还是有可能重新回到安全状态的，不过我们需要在分配资源之前总是考虑最坏的情况
  - 银行家算法
    - 进程提出资源申请的时候，先预判此次分配是否会导致系统进入不安全状态，如果会进入不安全状态，就暂时不答应这份请求
- 检测和解除
  - 检测
    - 死锁定理
      - 用某种数据结构来保存资源的请求和分配信息
      - 死锁定理提出：当且仅当系统资源的分配图中存在一个不可简化的环路（即循环等待）时，系统处于死锁状态
    - 资源分配图
      - 提供一种和数据结构相对应的算法，利用上述信息来检测系统是否进入了死锁状态
      - 资源节点指向进程节点代表分配边
      - 进程结点指向资源节点代表请求边
      - 请求边和分配边是可以相互抵消的，如果最终能消除所有边，那么就没有发生死锁，否则就可能发生死锁现象
  - 解除
    - 资源剥夺法
      - 挂起某些死锁进程并给之他的资源，同时要注意防止这些被挂起的进程长时间得不到资源而饿死
    - 撤销进程法
      - 强制撤销部分，甚至全部死锁进程，并剥夺这些进程的资源
      - 实现简单代价大
    - 进程回退法
      - 让一个或多个死锁进程回退到足以避免死锁的地步，这就要求系统记录进程的历史信息，设置还原点
      - 对症动手？
        - 进程优先级
        - 已执行多长时间
        - 还要多久完成
        - 进程已经使用了多少资源
        - 进程是交互式的还是批处理式的
- 死锁的计算题
  - 如果资源
    - 三个并发进程，每个进程需要4个资源，至少需要多少个资源？
    - $3*(4-1)+1=10$

进程调度概念

- 当有一堆任务需要处理，但是由于资源有限，这些事情没法同时处理，这就需要确定某种规则来决定处理这些任务的顺序，这就是调度研究的问题

进程调度的时机、切换与过程、方式

- 调度时机
  - 进程调度就是指将某种算法，从就绪队列中选择一个进程为其分配处理机
  - 进程正常终止
  - 运行过程中发生异常
  - 主动放弃
    - 等待事件发生
      - 进程主动请求阻塞
    - 分给进程的时间片用完了
  - 需要进程调度和切换的情况
    - I/O中断
      - 有更加紧急的事情要处理
    - 被动放弃
    - 有更高优先级的进程进入就绪队列
    - 处理中断
    - 在操作系统内核程序临界区
    - 不能进行进程切换的情况
    - 在原子操作过程中
- 调度程序本决定
  - 让渡运行
    - 调度算法
  - 运行时间
    - 时间片
- 进程调度的方式
  - 非抢占式的
  - 抢占式的
- 进程切换主要是为了
  - 1.对原来运行的进程各种数据的保存
  - 2.对新的进程各种数据的恢复
- 问题进程
  - 调度程序本身的激励，没有其他就绪进程时候，运行闲置进程

处理机调度层次

- 高级调度
  - 作业调度
    - 创建态
- 中级调度
  - 内存调度
    - 挂起态变成就绪态
    - 挂起的程序都在外存的交换区
    - 外存中由交换区和文件区两个区域组成
    - 按照某种策略决定将哪个早于挂起状态的进程重新调入内存，一个进程可能会多次调入调出，频率高于高级调度
- 低级调度
  - 进程调度
    - 就绪态变成执行态
    - 按照某种策略从就绪队列中选取一个进程，将处理及分配给他，频率很高几十秒一次

处理机调度概念

- 按照一定的原则从外存的作业后备队列中挑选一个作业调入内存，并创建进程，一个作业调入一次，调出一次
- 作业
  - 作业提交到作业完成期的时间
  - 周转时间
    - 作业
    - 进程
    - 进程进入就绪队列到完成时间
  - 平均周转时间
- 评价调度算法的指标
  - 带权周转时间
    - 作业周转时间/作业实际运转时间
  - 平均带权周转时间
  - 等待时间
    - 进程
    - 作业
    - 等待时间就是指进程建立之后等待被服务的时间之和
    - 等待时间越长调度度越低
    - 不仅要考虑建立进程的等待时间，还要加上作业在外村后备队列中等待的时间
  - 平均等待时间
  - 响应时间
    - 提交请求到首次产生相应所需要的时间
- 调度算法
  - 先来先服务
    - 作业一般就用这个
    - 作业调度进程调度都可以
    - 公平性
  - 高响应比优先
    - 非抢占式优先算法
    - 对长作业友好的
    - 等待时间+要求服务时间/要求服务时间
  - 时间片轮转
  - 优先级调度算法
    - 更偏向IO进程，资源利用率高
  - 多级反馈队列
    - 抢占式
  - 短作业优先/知道精优先
    - 非抢占式
  - 最短剩余时间优先
    - 抢占式
  - 只对短作业友好的
    - 短作业优先
    - 最短剩余时间优先
  - 调度算法分类
    - 只对长作业友好的
    - 先来先服务
    - 对短作业长作业都友好的
    - 多级反馈队列
    - 高响应比优先

进程同步和互斥

- 具体实现
- 进程同步
  - 进程同步也称为直接制约关系，它是指未完成某种任务而建立的两个或者多个进程，这些进程因为需要在某些位置上协调他们的工作次序而产生的制约关系。进程之间的直接制约关系就是源于他们之间的相互合作
- 进程互斥
  - 进程开发需要共享的支持，各个并发执行的进程不可避免的需要共享一些系统资源（比如内存，又如打印机，摄像头这样的I/O设备）
  - 互斥共享方式
    - 同时共享方式
    - 两种资源共享方式
      - 我们把在一个时间段内只允许一个进程使用的资源称为临界资源
      - 临界资源的访问必须得是互斥地进行
    - 临界资源
      - 对临界资源的访问代码可以从逻辑上对进程分成四个部分
        - 进入区
          - 负责检查是否可以进入临界区，如果可以进入，则应该设置正在访问临界资源的标志，以阻止其他进程同时进入临界区
          - 注意注意，这个各个区都是代码都是代码！
        - 临界区
          - 访问临界资源的那段代码
        - 退出区
          - 负责解除正在访问临界资源的标志
        - 剩余区
          - 做其他处理
      - 对临界资源的互斥访问需要遵循的四条原则***（理解就行）
        - 空闲让进
          - 临界区空闲的时候，可以允许一个请求进入临界区的进程立即进入临界区
        - 忙则等待
          - 当已有进程进入临界区的时候，其他试图进入临界区的进程必须等待
        - 有限等待
          - 对请求访问的进程，应保证能在有限时间内进入临界区
        - 让权等待
          - 当进程不能进入临界区的时候，应立即释放处理机，防止进程忙等待
  - 信号量
    - 信号量是指的，信号量的绝对值代表等待个数
    - 信号量考点
    - 在操作系统中，PV操作是一种低级进程通信原语
    - PV操作的理解

管程

- 子主题1

PV操作

- 类型分类
  - PPVV，这个代表服务之间存在缓冲区，服务和被服务是不在同一时间内实现的
    - 生产者消费者
    - 多生产者多消费者
    - 读者写者问题
      - 允许多个用户同时对临界资源访问的实现
      - 第一个人上锁
      - 最后一个人解锁
  - PVVP，这个代表服务是直接给到记的，服务和被服务是同时的
    - 理发师顾客问题
    - 银行顾客问题
    - 临界资源就是对这个等待序列的访问权限
      - 允许多个用户同时对临界资源访问的实现
      - 第一个人上锁
      - 最后一个人解锁
  - 哲学家进餐问题
- 信号量考点
  - 信号量是指的，信号量的绝对值代表等待个数
- PV操作的理解
  - 在操作系统中，PV操作是一种低级进程通信原语

进程调度的算法

- 先来先服务
  - 作业一般就用这个
  - 作业调度进程调度都可以
  - 公平性
- 高响应比优先
  - 非抢占式优先算法
  - 等待时间+要求服务时间/要求服务时间
- 时间片轮转
- 优先级调度算法
  - 更偏向IO进程，资源利用率高
- 多级反馈队列
  - 抢占式
- 短作业优先/知道精优先
  - 非抢占式
  - 短作业优先，长作业长时间
- 最短剩余时间优先
  - 抢占式
- 只对短作业友好的
  - 短作业优先
  - 最短剩余时间优先
- 只对长作业友好的
  - 先来先服务
- 对短作业长作业都友好的
  - 多级反馈队列
  - 高响应比优先

处理机

- 概念
  - 死锁是指多个进程（或线程）在执行过程中，因争夺资源而陷入互相等待的状态，导致所有进程都无法继续执行的情况
- 产生
  - 必要条件
    - 互斥
      - 只有对必须互斥使用的资源的争抢才会导致死锁，诸如：内存、磁盘驱动器、打印机等
    - 不剥夺
      - 进程所获得的资源在未使用之前，不能由其他进程强行夺走，只能主动释放
    - 请求和保持
      - 进程已经保持了至少一个资源，但又提出了新的资源请求，而该资源又被其他的进程所占有，此时请求进程被阻塞，但又对自己已有的资源保持不放
    - 循环等待
      - 存在一种进程资源的循环等待链，链中的每一个进程已获得的资源同时被下一个进程所请求
- 预防
  - 破坏互斥
    - 增加一个中间件
    - 如果把只能互斥使用的资源改造成允许共享使用的，那么系统就不会进入死锁状态
    - 缺点：并不是所有的互斥资源都可以改成共享使用的
  - 破坏不剥夺
    - 方案1
      - 当某个进程请求新的资源得不到满足的时候，它必须立即释放保持的所有资源，待以后需要的时候再重新申请。也就是说，即使某些资源尚未使用完也需要主动释放
      - 缺点
        - 实现复杂
        - 释放已经获得的资源一般只适用于易于恢复状态的资源
        - 反复地申请和释放资源会增加系统开销，降低系统吞吐量
        - 如果采用方案1，如果暂时地得不到某个资源之前获得的全部资源都要释放掉，如果一直发生这种的情况会导致饥饿
    - 方案2
      - 当某个进程需要的资源被其他进程占有的时候，可以由操作系统协助，将想要的资源强行剥夺，这种方式一般需要考虑进程的优先级
  - 破坏请求与保持条件
    - 可以采用静态分配的方法
    - 一次性分配全部需要的资源
      - 缺点
        - 缺点资源利用率较低，一旦占有就一直占有，CPU运行的时候，也基于不了其他事，效率很低
        - 实现比较简单
  - 破坏循环等待条件
    - 顺序资源分配的方法
      - 1.给系统的资源编号
      - 2.每个进程必须按照编号递增的顺序请求资源，同类资源一次申请完
      - 原理就是不给进程申请到的资源的编号比我小，它可以申请我的资源，但是我的编号已经过了他的资源的编号，那么我就不能申请他的资源了，从而打破了循环等待的过程
      - 缺点
        - 不方便增加新的设备，因为可能会把所有设备的编号全部修改
        - 进程实际使用资源的顺序可能和编号递增顺序不一致，导致资源的浪费
        - 必须按照规定次序申请资源，用户编程麻烦
- 避免
  - 安全序列
    - 就是指如果系统按照这种序列分配资源，则每个进程都能顺利完成。只要能找出一个安全序列，系统就是安全状态。当把安全序列写错很多个
    - 一定不会发生死锁
  - 不安全状态
    - 可能会发生死锁
    - 分配了资源之后，如果找不到任何一个安全序列，那么当前就处于不安全状态。这就意味着所有进程都无法顺利的执行下去
    - 但是如果有一些进程能顺利结束了资源，还是有可能重新回到安全状态的，不过我们需要在分配资源之前总是考虑最坏的情况
  - 银行家算法
    - 进程提出资源申请的时候，先预判此次分配是否会导致系统进入不安全状态，如果会进入不安全状态，就暂时不答应这份请求
- 检测和解除
  - 检测
    - 死锁定理
      - 用某种数据结构来保存资源的请求和分配信息
      - 死锁定理提出：当且仅当系统资源的分配图中存在一个不可简化的环路（即循环等待）时，系统处于死锁状态
    - 资源分配图
      - 提供一种和数据结构相对应的算法，利用上述信息来检测系统是否进入了死锁状态
      - 资源节点指向进程节点代表分配边
      - 进程结点指向资源节点代表请求边
      - 请求边和分配边是可以相互抵消的，如果最终能消除所有边，那么就没有发生死锁，否则就可能发生死锁现象
  - 解除
    - 资源剥夺法
      - 挂起某些死锁进程并给之他的资源，同时要注意防止这些被挂起的进程长时间得不到资源而饿死
    - 撤销进程法
      - 强制撤销部分，甚至全部死锁进程，并剥夺这些进程的资源
      - 实现简单代价大
    - 进程回退法
      - 让一个或多个死锁进程回退到足以避免死锁的地步，这就要求系统记录进程的历史信息，设置还原点
      - 对症动手？
        - 进程优先级
        - 已执行多长时间
        - 还要多久完成
        - 进程已经使用了多少资源
        - 进程是交互式的还是批处理式的
- 死锁的计算题
  - 如果资源
    - 三个并发进程，每个进程需要4个资源，至少需要多少个资源？
    - $3*(4-1)+1=10$